

multiThreading intrevwei question with details answer

Here's a complete list of important Java Multithreading Interview Questions with detailed and easy-to-understand answers — from basics to advanced.

1. What is Multithreading in Java?

Answer:

Multithreading is the ability of a program to execute multiple parts (threads) of a process simultaneously. Each thread runs independently but shares the same memory space.

Example:

java

 Copy code

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println(Thread.currentThread().getName() + " is running");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
        MyThread t2 = new MyThread();  
        t1.start();  
        t2.start();  
    }  
}
```

Output:

arduino

 Copy code

```
Thread-0 is running  
Thread-1 is running
```

Key Point:

Multithreading improves performance but increases complexity due to thread synchronization issues.



2. Difference between Process and Thread

Feature	Process	Thread
Memory	Has its own memory	Shares memory with other threads
Communication	Through Inter-process Communication (IPC)	Easier, via shared memory
Creation Cost	Heavy	Lightweight
Dependency	Independent	Dependent on process

3. How do you create a Thread in Java?

Two ways:

1. Extending Thread class
2. Implementing Runnable interface

Example 1 (Extending Thread):

java

 Copy code

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread running using Thread class");  
    }  
}
```

Example 2 (Implementing Runnable):

java

 Copy code

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Thread running using Runnable");  
    }  
}
```

Why prefer Runnable?

- Java supports single inheritance, so using Runnable allows extending another class.

4. What's the difference between start() and run() method?

Method	Behavior
start()	Creates a new thread and calls the run() method in that thread.
run()	Executes in the same thread (no new thread created).

Example:

java

 Copy code

```
Thread t = new Thread(() -> System.out.println(Thread.currentThread().getName()))  
t.run(); // runs in main thread  
t.start(); // runs in new thread
```

5. What is Thread Lifecycle in Java?

Thread states:

1. **New** – Thread created but not started.
2. **Runnable** – Ready to run or running.
3. **Blocked/Waiting** – Waiting for a monitor lock or condition.
4. **Timed Waiting** – Waiting for a specific time.
5. **Terminated** – Execution finished.

6. What is Synchronization?

Synchronization prevents multiple threads from accessing a shared resource simultaneously — avoids data inconsistency.



Example:

java

 Copy code

```
class Counter {  
    private int count = 0;  
  
    public synchronized void increment() {  
        count++;  
    }  
  
    public int getCount() {  
        return count;  
    }  
}
```

Without `synchronized`, multiple threads may increment the same variable simultaneously → inconsistent result.

7. What is Deadlock?

A deadlock occurs when two or more threads are waiting for each other's resources, so none can proceed.

Example:

java

 Copy code

```
class A {  
    synchronized void methodA(B b) {  
        System.out.println("Thread1 locked A, waiting for B");  
        try { Thread.sleep(100); } catch(Exception e) {}  
        b.last();  
    }  
  
    synchronized void last() {}  
}
```

```
}
```

```
class B {  
    synchronized void methodB(A a) {  
        System.out.println("Thread2 locked B, waiting for A");  
        try { Thread.sleep(100); } catch(Exception e) {}  
        a.last();  
    }  
    synchronized void last() {}  
}
```

Avoid by:

- Always acquiring locks in the same order.
- Using `tryLock()` from `ReentrantLock`.

✖ 8. What is volatile keyword?

`volatile` ensures that a variable's value is always read from main memory and not from thread cache.

Example:

```
java
```

[Copy code](#)

```
volatile boolean flag = false;
```

Without `volatile`, one thread may not see changes made by another.

🧱 9. What is join() method?

Makes one thread wait until another thread finishes.

Example:

```
java
```

[Copy code](#)

```
Thread t1 = new Thread(() -> System.out.println("Task 1 done"));  
t1.start();
```

```
t1.join(); // main waits for t1
System.out.println("Main continues");
```

10. What is Thread Priority?

Threads have priorities (1–10). The scheduler uses it as a hint to decide execution order.

java

 Copy code

```
t1.setPriority(Thread.MAX_PRIORITY);
t2.setPriority(Thread.MIN_PRIORITY);
```

 Priority doesn't guarantee order — depends on OS thread scheduler.

11. What is Executor Framework?

Introduced in Java 5, it simplifies thread management — instead of manually creating threads.

Example:

java

 Copy code

```
ExecutorService executor = Executors.newFixedThreadPool(2);
executor.submit(() -> System.out.println("Task executed"));
executor.shutdown();
```

Benefits:

- Thread reusability
- Easier to manage lifecycle
- Built-in queue handling

12. What is Callable and Future?

Callable is like Runnable but returns a result and can throw checked exceptions.

Example:

java

 Copy code

```
Callable<Integer> task = () -> 5 + 10;
```

```
ExecutorService executor = Executors.newSingleThreadExecutor();  
Future<Integer> future = executor.submit(task);
```

```
System.out.println(future.get()); // Output: 15  
executor.shutdown();
```

 13. What is the difference between synchronized and Lock ?

Feature	synchronized	Lock (ReentrantLock)
Type	Keyword	Interface in <code>java.util.concurrent.locks</code>
Flexibility	Automatic release	Must release manually
Try Locking	Not possible	Possible via <code>tryLock()</code>
Fairness	Not guaranteed	Can be made fair

Example:

java

 Copy code

```
Lock lock = new ReentrantLock();  
lock.lock();  
try {  
    // critical section  
} finally {  
    lock.unlock();  
}
```



14. What is a Thread Pool?

A thread pool is a group of pre-instantiated threads ready to execute tasks.

Managed by `ExecutorService`.



Example:

java

Copy code

```
ExecutorService pool = Executors.newFixedThreadPool(3);
for (int i = 0; i < 5; i++) {
    pool.execute(() -> System.out.println(Thread.currentThread().getName()));
}
pool.shutdown();
```



15. What is `wait()`, `notify()`, and `notifyAll()`?

Used for inter-thread communication.

Example:

java

Copy code

```
synchronized(obj) {
    obj.wait();    // waits until notified
}

synchronized(obj) {
    obj.notify();  // wakes one waiting thread
}
```

Important:

- Must be called inside a synchronized block.
- Otherwise throws `IllegalMonitorStateException`.



16. What is the difference between `sleep()` and `wait()`?

Feature	sleep()	wait()
Belongs to	Thread class	Object class
Lock	Doesn't release lock	Releases lock
Resume	After timeout	When notified
Usage	Pause thread	Communication between threads

17. What is ConcurrentHashMap and how it differs from HashMap?

- HashMap is **not thread-safe**.
- ConcurrentHashMap is **thread-safe** without locking the entire map.

Internally: Uses **segments** and **fine-grained locking** to allow multiple threads to operate on different parts concurrently.

18. What is the difference between ThreadLocal and synchronization?

- synchronized : Shares one variable safely between threads.
- ThreadLocal : Gives **each thread its own copy** of a variable.

Example:

java

 Copy code

```
ThreadLocal<Integer> threadId = ThreadLocal.withInitial(() -> 0);
```

19. How to handle exceptions in threads?

Use `Thread.setUncaughtExceptionHandler()` .

Example:

java

 Copy code

```
Thread t = new Thread(() -> { throw new RuntimeException("Error"); });  
t.setUncaughtExceptionHandler((th, e) -> System.out.println("Exception: " + e));  
t.start();
```

20. What are best practices for Multithreading?

- ✓ Keep critical sections small
- ✓ Avoid nested locks
- ✓ Prefer higher-level concurrency utilities (Executor , CountdownLatch , Semaphore)
- ✓ Use immutable objects when possible
- ✓ Always release locks in finally block

Would you like me to create a **PDF cheat sheet** of these 20 multithreading questions